

HASHEDIN BY DELOITTE

Engineering Learning & Development

**Pre-Requisite Foundations:
Python · FastAPI · Generative AI Basics**

Foundation Course Content — Required Before the Multi-Agent Systems Engineering Program

Field	Value
Document	MAS_TRAINING-000_Prerequisite_Foundations
Audience	Engineers preparing to enter the Multi-Agent Systems Engineering program
Author	Nabeel Qazi
Owner	Nabeel Qazi
Version	1.0
Status	Internal Use

Document Control

Field	Detail
Document Title	Pre-Requisite Foundations — Python, FastAPI & Generative AI Basics
When to Use	Before enrolling in MAS-TRAINING-001 (Multi-Agent Systems Engineering). Engineers who already meet every item in the Prerequisites table may test out.
Companion Docs	PBEG-AI-TRAINING-004 (Evaluator Guide — graders only) · MAS-TRAINING-001 (Mandatory Content) · MAS-TRAINING-003 (Assignment Handbook)
Scope	Python fundamentals · FastAPI service development · Generative AI basics (tokens, embeddings, prompt engineering, LLM APIs). Django, RAG pipelines and full agentic apps are out of scope and covered later in the MAS program.
Assignments	5 graded foundation assignments · 10 marks each · 50 marks total · all Medium difficulty
Total Study Time	~30–35 hours of guided study across 3 tracks
Author	Nabeel Qazi · Engineering Learning & Development

Course durations sourced from official course pages where available.

Udemy and corporate-portal durations reflect the platform listing — verify current length when accessing. Items marked “(verify on [platform])” could not be independently confirmed at authoring time.

Introduction

This document is the on-ramp to the Multi-Agent Systems Engineering program. The MAS curriculum assumes engineers are already comfortable writing Python, building and testing a small web service, and reasoning about how large language models consume and produce text. This foundation course brings every engineer to that baseline before Module 1 of MAS-TRAINING-001.

It is organised into three tracks that build on each other: Python fundamentals establish the language and tooling, FastAPI turns that into the ability to ship a real HTTP service, and Generative AI basics give the vocabulary and hands-on reps with tokens, embeddings, and prompt engineering that the agent work depends on. Five graded assignments — all Medium difficulty — confirm readiness across all three areas.

Who This Is For

- Engineers joining the MAS program who have written some Python but have not built and tested a production-style API.

- Engineers comfortable with backend work who have not yet worked directly with LLM APIs, tokens, or structured prompting.
- Anyone who wants to confirm they meet the MAS prerequisites before the intensive begins.

How to Use This Document

Work the three tracks in order. Each lesson lists verified resources with the exact sections to cover and the sections to skip. Complete the five assignments as you go — Assignment 1 after the Python track, Assignments 2–3 during the FastAPI track, and Assignments 4–5 during the GenAI track. The assignments are cumulative: the GenAI assignments are built as FastAPI endpoints, so the service you build early is reused later.

Contents of This Document

Track	Focus	Time
Track 1 — Python Foundations	Syntax, data structures, typing, errors, JSON, virtual environments, pytest	~10 hrs
Track 2 — FastAPI & Web Services	HTTP, REST, Pydantic, routing, dependencies, auth, testing	~11 hrs
Track 3 — Generative AI Basics	Tokens, embeddings, LLM APIs, prompt engineering, structured outputs	~11 hrs
Assignments 01–05	5 Medium graded assignments spanning all three tracks	~13 hrs

Pre-requisites for This Foundation Course

This is an entry course, so the bar is low. Engineers should arrive with the following. Where a gap exists, remediate it using Track 1 before moving on.

Area	What you should already have
Programming basics	Have written code in some language. Understand variables, loops, conditionals, and functions as concepts even if not yet fluent in Python syntax.
Command line	Can open a terminal, navigate directories, and run a command. Comfortable installing software.
Editor	A working code editor (VS Code recommended) and the ability to create, save, and run a file.
Accounts	A GitHub account and the ability to obtain an OpenAI (or equivalent) API key. Billing/credits arranged before the GenAI track.

Curriculum at a Glance

Three tracks, progressing from language fundamentals to a working LLM-backed service. Assignments are interleaved so each new skill is exercised immediately.

#	Track / Lesson	Time	Focus
1	Python Foundations	~10 hrs	Language, tooling, errors, JSON, testing
2	FastAPI & Web Services	~11 hrs	HTTP, REST, Pydantic, routing, auth, tests
3	Generative AI Basics	~11 hrs	Tokens, embeddings, LLM APIs, prompting

#	Assignment	Primary Track	Difficulty	Hours
01	Data Toolkit CLI	Python	Medium	~2.5 hrs
02	Weather & Jokes API Service	FastAPI	Medium	~3 hrs
03	Secured Notes API with Tests	FastAPI	Medium	~3 hrs
04	Prompt Engineering Toolkit	GenAI	Medium	~2.5 hrs
05	LLM Summariser & Classifier Service	GenAI + FastAPI	Medium	~3 hrs

Track 1 — Python Foundations

Python is the language of the entire MAS program. This track gets engineers fluent enough to read and write idiomatic Python — data structures, typed functions, error handling, JSON, virtual environments, and pytest. Already fluent? Skim and go straight to Assignment 01.

Python 3.10+ is required across all tracks. Use a virtual environment (venv) for every assignment — never install globally.

Modern type-hint syntax (`list[str]`, `str | None`) and the `match` statement assume 3.10+.

Estimated time: ~10 hours · three lessons

Lesson 1.1 — Syntax & Data Structures

Outcome: use the core types (`str`, `int`, `float`, `bool`, `list`, `dict`, `set`, `tuple`) fluently; write functions with default and keyword arguments; transform collections with comprehensions.

Resources

- [Python for Beginners — freeCodeCamp](#) (~4.5 hrs — verify in browser)
 - ↳ First ~2 hours: variables, types, lists, dicts, functions (priority for Assignment 01). Skip Tkinter/GUI and the scraping capstone.
- [Google's Python Class \(official, text + exercises\)](#) (~2 hrs)
 - ↳ Strings, Lists, Dicts, Files — do the exercises, don't just read.

Lesson 1.2 — Errors, Typing & JSON

Outcome: handle exceptions with `try/except` and raise clear custom errors; annotate functions with type hints (`list[str]`, `str | None`) and explain why they aid tooling; convert between Python objects and JSON with `json.loads` / `json.dumps`.

Resources

- [Python Exceptions — Real Python](#) (~30 mins)
 - ↳ Raising, catching, and the exception hierarchy.
- [Python Type Checking — Real Python](#) (~45 mins)
 - ↳ “Hello Types”, function annotations, the typing module. Skip protocols/ABCs/runtime checking.
- [Working With JSON Data in Python — Real Python](#) (~30 mins)
 - ↳ `json.loads` / `json.dumps` and serialising nested dicts.

Lesson 1.3 — Environments & Testing

Outcome: create and activate a `venv` and pin dependencies in `requirements.txt`; split a project into importable modules; write and run `pytest` unit tests with assertions and simple fixtures.

Resources

- [Python Virtual Environments: A Primer — Real Python](#) (~30 mins)
 - ↳ Creating, activating, and why isolation matters.
- [Getting Started With pytest — Real Python](#) (~1 hr)
 - ↳ Test functions, assertions, and fixtures (priority for Assignment 03). Skip the plugin ecosystem for now.

Track 2 — FastAPI & Web Services

Every agent the MAS program builds eventually runs behind an HTTP boundary — the A2A assignment is literally two FastAPI services talking to each other. This track builds the ability to design, run, document, secure, and test a web service. It opens with the HTTP and REST concepts the rest of the track assumes, then moves into FastAPI proper.

FastAPI is the required web framework for this foundation course and for the A2A work in MAS.

Use Pydantic models for all request and response bodies — do not hand-parse JSON. Run the dev server with the fastapi dev command and use the auto-generated /docs (Swagger UI) to exercise every endpoint.

Estimated time: ~11 hours

Lesson 2.1 — HTTP, REST & Calling APIs

Learning Outcomes

- Explain the HTTP request/response cycle: methods (GET, POST, PUT, DELETE), status codes, headers, and bodies.
- Describe what makes an API RESTful: resources, verbs, and statelessness.
- Call an external API from Python with requests/httpx and parse the JSON response.

Resources

- [HTTP Crash Course — Traversy Media](#) (~1 hr — duration unverified, check in browser)
↳ **Methods, status codes, request/response anatomy**
- [What is a REST API? — concepts video](#) (~1 hr — duration unverified, check in browser)
- [Requests: Quickstart \(official docs\)](#) (~30 mins)

↳ **Making requests, query params, JSON responses, status codes**

Practice APIs (no key needed for the second):

- [OpenWeather API](#) — free key, used in Assignment 02
- [JokeAPI](#) — no key required, used in Assignment 02

Lesson 2.2 — FastAPI Fundamentals

Learning Outcomes

- Build path and query parameter endpoints and return JSON automatically.
- Define request and response schemas with Pydantic models and rely on automatic validation.
- Use the auto-generated OpenAPI docs (/docs) to test endpoints interactively.

Resources

- [FastAPI Tutorial — User Guide \(official docs\)](#) (~4 hrs, worked end-to-end)
↳ **First Steps → Path Parameters → Query Parameters → Request Body → Response Model (priority for Assignments 02–03)**
X Skip: Advanced User Guide, WebSockets, and SQL database sections — out of scope for this foundation course.
- [FastAPI Tutorial — Tech With Tim](#) (~1 hr — duration unverified, check in browser)

↳ Use as a guided first pass before working the official docs.

Lesson 2.3 — Dependencies, Auth, Logging & Testing

Learning Outcomes

- Use FastAPI dependency injection (Depends) to share logic such as auth checks across endpoints.
- Protect endpoints with API-key or token-header authentication and return correct 401/403 responses.
- Add structured logging and write endpoint tests with FastAPI's TestClient and pytest.

Resources

- [FastAPI — Dependencies \(official docs\)](#) (~45 mins)
 - ↳ **First Steps and Classes as Dependencies**
- [FastAPI — Security: First Steps \(official docs\)](#) (~45 mins)
 - ↳ **API-key / bearer-token header pattern and 401 handling (priority for Assignment 03)**
 - ✗ **Skip:** Full OAuth2 password flow with hashing and JWT — only skim; not required for the foundation assignments.
- [FastAPI — Testing \(official docs\)](#) (~30 mins)
 - ↳ **TestClient, asserting status codes and JSON bodies**
- [Python Logging \(official docs\)](#) (~30 mins)
 - ↳ Basic configuration, log levels, and logging to console

Track 3 — Generative AI Basics

This track gives engineers the conceptual vocabulary and hands-on reps the MAS program assumes from day one: what a token is and why it governs cost and context limits, how embeddings turn text into vectors, how to call an LLM API from Python, and — most importantly — how to engineer prompts that produce reliable, structured output. No agents or RAG pipelines yet; those are built in the MAS program itself.

You need a working LLM API key (OpenAI or equivalent) for this track.

Set a small spending limit before you start. Load the key from a .env file via environment variables — never hardcode a key in source and never commit it to git. This habit is graded in the MAS program.

Estimated time: ~11 hours

Lesson 3.1 — Tokens, Embeddings & How LLMs Work

Learning Outcomes

- Explain what a token is, how tokenisation works, and why tokens drive both cost and context-window limits.
- Describe what an embedding is and how vector similarity captures semantic closeness.
- Reason at a high level about how a transformer-based LLM predicts the next token.

Resources

- [OpenAI — What are tokens and how to count them \(Help Center\)](#) (~20 mins reading)
 - ↳ **Token-to-word ratios and why long prompts cost more**
- [OpenAI Tokenizer \(interactive tool\)](#) (~10 mins, hands-on)
 - ↳ **Paste several texts and watch the token count change**
- [What are Word Embeddings? — IBM Technology](#) (~20 mins — duration unverified, check in browser)
- [Introduction to Large Language Models — Google Cloud](#) (~20 mins — duration unverified, check in browser)
 - ↳ High-level model of training and next-token prediction

Lesson 3.2 — Calling LLM APIs

Learning Outcomes

- Call a chat-completions API from Python with system and user messages.
- Control output with parameters: temperature, max_tokens, and stop sequences.
- Request and parse structured (JSON) output from the model reliably.

Resources

- [OpenAI — Developer Quickstart \(official docs\)](#) (~45 mins, hands-on)
 - ↳ **Make your first call; understand the messages array and the response object**
- [OpenAI — Text generation & prompting \(official docs\)](#) (~45 mins)
 - ↳ **System vs user roles, temperature, and max output tokens**
- [OpenAI — Structured Outputs \(official docs\)](#) (~45 mins)

↳ **Forcing JSON output and validating it against a schema (priority for Assignments 04–05)**

Lesson 3.3 — Prompt Engineering

Learning Outcomes

- Apply the core prompting principles: write clear and specific instructions, and give the model time to “think”.
- Use roles, delimiters, few-shot examples, and step-by-step instructions to shape output.
- Iterate on a prompt systematically, diagnosing why an output missed and adjusting one variable at a time.

Resources

- [ChatGPT Prompt Engineering for Developers — DeepLearning.AI](#) *1h 30m · 9 video lessons*
 - ↳ **Guidelines, Iterative, Summarizing, Inferring, Transforming, Expanding (priority for Assignment 04)**
 - ↳ Confirmed: 1.5 hours, 9 lessons, free on the DeepLearning.AI platform.
- [Prompt Engineering Guide \(promptingguide.ai\)](#) *(~1.5 hrs reading)*
 - ↳ **Basics, Elements of a Prompt, and Techniques: Few-shot and Chain-of-Thought**
 - ✗ **Skip:** Adversarial prompting, RAG, and agent sections — covered later in the MAS program.
- [OpenAI — Prompt engineering best practices \(official docs\)](#) *(~30 mins)*
 - ↳ Six strategies for getting better results

Foundation Assignments

Five graded assignments, all Medium difficulty, total 50 marks. Each assignment is marked on the standard 5-criterion rubric (2 marks each). The assignments are cumulative: the service you build in Assignment 02 is the foundation for Assignment 03, and the prompting and LLM-API skills from Assignments 04–05 are exactly what the MAS program builds agents on top of.

#	Title	Track	Marks	Hours
01	Data Toolkit CLI	Python	10	~2.5 hrs
02	Weather & Jokes API Service	FastAPI	10	~3 hrs
03	Secured Notes API with Tests	FastAPI	10	~3 hrs
04	Prompt Engineering Toolkit	GenAI	10	~2.5 hrs
05	LLM Summariser & Classifier Service	GenAI + FastAPI	10	~3 hrs

Pass criteria for the foundation course: 30/50 overall, with at least 12/30 across Assignments 01–03 and at least 8/20 across Assignments 04–05.

Standard rubric: Criterion 1 (Structure & Correctness) · Criterion 2 (Core Functionality) · Criterion 3 (Error Handling & Robustness) · Criterion 4 (End-to-End Run) · Criterion 5 (Documentation). Framework substitutions where a tool is required (e.g. not using FastAPI in 02–03) lose Criterion 1 marks.

Assignment 01 — Data Toolkit CLI

Difficulty	Marks	Est. Hours	Required Stack
MEDIUM	10 marks	~2.5 hrs	Python 3.10+ · standard library · pytest
Pattern: Python Foundations — modules, typed functions, error handling, JSON I/O, unit tests			

Scenario

An engineering team keeps its sprint data in a messy JSON file exported from a planning tool. Analysts waste time hand-counting tasks and recomputing totals. You are building a small command-line toolkit that loads the file, validates it, and answers a handful of common questions — the kind of reliable, typed, tested utility every engineer should be able to produce before touching agents.

What You Need to Build

A CLI script, `toolkit.py`, plus a `tasks.json` data file, organised into a small package with at least one importable module (e.g. `analytics.py`) that `toolkit.py` calls.

Input data

Commit a `tasks.json` containing a list of at least 12 task objects. Each task has: `id` (int), `title` (str), `assignee` (str), `story_points` (int), `status` (one of 'todo' / 'in_progress' / 'done' / 'blocked'), and `tags` (list of str).

Functions (in the importable module, all type-hinted)

- **`load_tasks(path: str) -> list[dict]`**: Reads and parses the JSON file. Raises a clear, custom error message if the file is missing or the JSON is malformed.
- **`count_by_status(tasks: list[dict]) -> dict[str, int]`**: Returns a count of tasks per status, e.g. `{'todo': 4, 'done': 5, ...}`.
- **`total_points(tasks: list[dict], status: str | None = None) -> int`**: Sums `story_points` across all tasks, or only those matching `status` when provided.
- **`assignee_load(tasks: list[dict]) -> dict[str, int]`**: Returns total open (non-done) story points per assignee, sorted highest first.
- **`filter_by_tag(tasks: list[dict], tag: str) -> list[dict]`**: Returns every task whose tags list contains the given tag (case-insensitive).

CLI behaviour

- Invoked as: `python toolkit.py <command> [args]`. Commands: `summary`, `points [status]`, `load [assignee]`, `tag <name>`.
- Unknown command or missing required argument prints a helpful usage message and exits with a non-zero status code — it does not crash with a traceback.
- Output is human-readable text, not a raw dict dump.

Constraints

- Every public function has type hints and a one-line docstring.
- All file and parsing errors are caught and reported as clear messages.

- No third-party packages — standard library only (argparse or sys.argv is fine).

Milestones

#	Phase	What You're Building	Time
M1	Project & Data Setup	Create the package layout, the tasks.json fixture, and the virtual environment with a requirements.txt (pytest only).	20 min
M2	Core Functions	Implement the five typed analytics functions in the importable module with docstrings.	45 min
M3	CLI & Error Handling	Wire the command dispatcher, usage messages, and exit codes; catch all file/JSON errors.	40 min
M4	Testing & Docs	Write pytest tests for each function (including an error case); README with run examples for all four commands.	30 min

Marking Rubric (10 marks)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
1. Structure & Correctness	Code split into an importable module + CLI; all functions type-hinted with docstrings; logic correct	Works but everything in one file, or type hints/docstrings largely missing	Functions absent or return wrong values; no module structure
2. Core Functionality	All five functions return correct results across the sample data and evaluator inputs	3–4 functions correct; one returns wrong counts/totals	Most functions broken or missing
3. Error Handling & Robustness	Missing file, bad JSON, unknown command, and missing args all produce clear messages and non-zero exit — no tracebacks	Some errors handled; at least one path crashes with a raw traceback	No error handling; crashes on any bad input
4. End-to-End Run	All four CLI commands run correctly on the committed data and evaluator data	3 of 4 commands correct; one wrong or errors	Crashes or wrong output on sample
5. Documentation	README with run examples for every command; tasks.json committed; pytest tests pass	Code runs; README missing examples or some tests absent	No README; no tests; no sample data

Submission Checklist

- tasks.json with at least 12 tasks committed
- Importable analytics module separate from the CLI entry point
- pytest tests covering each function and at least one error case
- README with example invocations and output for all four commands

Assignment 02 — Weather & Jokes API Service

Difficulty	Marks	Est. Hours	Required Stack
MEDIUM	10 marks	~3 hrs	Python · FastAPI · httpx/requests · Pydantic
Pattern: FastAPI — typed endpoints, external API calls, Pydantic response models, error handling			

Scenario

A team wants a single internal service that wraps a couple of noisy third-party APIs behind clean, documented, validated endpoints — so the rest of the team never has to handle raw upstream JSON or leaked API keys again. You are building that gateway service with FastAPI, the same framework the MAS program uses for its agent-to-agent work.

What You Need to Build

A FastAPI application, `main.py`, exposing three endpoints. All responses are typed with Pydantic models. The upstream weather key is loaded from the environment, never hardcoded.

Endpoints

Method	Path	Description
GET	/health	Returns {"status": "ok"} — a liveness check.
GET	/weather?city={city}	Calls the OpenWeather API for the city and returns a trimmed, typed response: city, temp_c, conditions.
GET	/joke	Calls JokeAPI and returns a typed response: setup and delivery (or a single-line joke normalised into the same shape).

Behaviours & constraints

- **Pydantic models:** Define `WeatherResponse` and `JokeResponse` models. FastAPI returns them as the `response_model` so the schema appears in `/docs`.
- **Upstream errors:** If OpenWeather returns a 404 for an unknown city, your service returns HTTP 404 with a clear JSON detail — not a 500. If the upstream is unreachable, return 502 with a clear message.
- **Missing param:** A request to `/weather` with no city returns 422 (FastAPI validation) — demonstrate this in the README.
- **Secrets:** The OpenWeather API key is read from an environment variable (e.g. via a `.env` loaded with `os.environ`). The key is never committed; `.env` is in `.gitignore` and a `.env.example` is provided.
- **Docs:** All three endpoints appear in the auto-generated `/docs` with correct response schemas.

Sample requests to cover in README

- GET /weather?city=London → 200 with temp and conditions
- GET /weather?city=NotARealCity → 404 with clear detail
- GET /weather (no city) → 422 validation error
- GET /joke → 200 with setup/delivery

Milestones

#	Phase	What You're Building	Time
M1	Service Skeleton	FastAPI app, virtual environment, /health endpoint, and Pydantic response models defined.	30 min
M2	External API Integration	Wire /weather and /joke to the upstream APIs via httpx/requests; map upstream JSON into your models.	50 min
M3	Errors & Secrets	Map upstream failures to correct status codes; load the key from the environment; add .env.example.	40 min
M4	Testing & Docs	Exercise all endpoints via /docs; README with the four sample requests and run command.	30 min

Marking Rubric (10 marks)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
1. Structure & Correctness	FastAPI app with all 3 endpoints; Pydantic response_models used; runs with fastapi dev and appears in /docs	Endpoints work but no Pydantic models, or a framework other than FastAPI used (– 1)	Not a working FastAPI app; endpoints missing
2. Core Functionality	/weather and /joke return correct trimmed, typed data from the live upstreams	One endpoint works; the other returns raw upstream JSON or wrong fields	Neither external endpoint returns correct data
3. Error Handling & Robustness	Unknown city → 404; unreachable upstream → 502; missing param → 422; all with clear detail	Some error paths handled; at least one returns a 500 / leaks a traceback	No error mapping; upstream failures crash the service
4. End-to-End Run	All four README sample requests behave exactly as specified	3 of 4 behave correctly	Service crashes or core requests fail
5. Documentation	README with run command + 4 sample requests; .env.example	Runs; README thin or .env.example missing	No README; or API key committed to git (also a red flag)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
	present; key not committed		

Submission Checklist

- FastAPI app with /health, /weather, /joke and Pydantic response models
- API key loaded from environment; .env in .gitignore; .env.example committed
- README with run command and the four sample requests + responses
- All endpoints visible with correct schemas in /docs

Assignment 03 — Secured Notes API with Tests

Difficulty	Marks	Est. Hours	Required Stack
MEDIUM	10 marks	~3 hrs	Python · FastAPI · Pydantic · pytest · Depends
Pattern: FastAPI — CRUD, dependency injection, API-key auth, pagination, TestClient tests			

Scenario

An internal tool needs a small notes backend that other services can read and write — but only with a valid API key, and without dumping thousands of records in a single response. You are building a secured, paginated, tested CRUD service. This is the closest foundation assignment to the production hygiene the MAS program expects: auth via dependencies, correct status codes, and real tests.

What You Need to Build

A FastAPI application exposing CRUD endpoints over an in-memory notes store (a dict or list — no database needed). Writes require authentication via an API-key header, enforced through a FastAPI dependency. Listing is paginated.

Data model

A Note has: id (int, server-assigned), title (str, required), body (str, required), and created_at (ISO timestamp, server-set). Use Pydantic models: NoteCreate (input) and Note (output).

Endpoints

Method	Path	Auth	Description
POST	/notes	Required	Create a note from NoteCreate; returns the created Note with 201.
GET	/notes?limit={n}&offset={m}	None	List notes, paginated; returns items plus total count.
GET	/notes/{id}	None	Retrieve one note; 404 if not found.
DELETE	/notes/{id}	Required	Delete a note; 204 on success, 404 if not found.

Behaviours & constraints

- **Auth dependency:** A single Depends-injected function checks an X-API-Key header against a key from the environment. Missing/wrong key → 401. The same dependency guards both write endpoints — no copy-pasted checks.
- **Pagination:** limit defaults to 10 (max 100), offset defaults to 0. The list response has shape {items: [...], total: int, limit: int, offset: int}.
- **Validation:** Creating a note with an empty title or missing body returns 422 automatically via Pydantic.

- **Status codes:** 201 on create, 204 on delete, 404 for missing id, 401 for bad/missing key, 422 for validation.
- **Tests:** pytest tests using FastAPI's TestClient covering: authed create succeeds, unauthed create is 401, list pagination works, get-missing is 404, delete works.

Milestones

#	Phase	What You're Building	Time
M1	Models & Store	Define NoteCreate / Note models and the in-memory store with server-assigned ids and timestamps.	30 min
M2	CRUD Endpoints	Implement create, list (paginated), get, and delete with correct status codes.	45 min
M3	Auth Dependency	Build the X-API-Key Depends function and guard both write endpoints; wire the key from the environment.	40 min
M4	Tests & Docs	Write the five TestClient tests; README with run command, auth instructions, and sample requests.	35 min

Marking Rubric (10 marks)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
1. Structure & Correctness	FastAPI CRUD with Pydantic input/output models; ids and timestamps server-assigned; correct status codes throughout	CRUD works but status codes wrong (e.g. 200 on create), or models conflated	Not CRUD; missing endpoints; wrong framework
2. Core Functionality (Auth + Pagination)	Auth enforced via a single Depends on both writes; 401 on bad key; pagination returns correct items + total	Auth present but duplicated inline rather than a dependency, or pagination ignores limit/offset	No auth, or writes are open; no pagination
3. Error Handling & Robustness	Missing id → 404; bad/missing key → 401; invalid body → 422; limit capped at 100	Most handled; one wrong (e.g. missing id returns 500)	No error handling; crashes on missing id or bad input
4. End-to-End Run	All sample requests behave as specified; full create→list→get→delete cycle works	Cycle mostly works; one step wrong	Crashes or core cycle fails

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
5. Documentation & Tests	Five TestClient tests pass; README with run + auth instructions; .env.example committed	Runs; some tests missing or README thin	No tests; no README

Submission Checklist

- FastAPI CRUD service with in-memory store and Pydantic models
- Single Depends auth dependency guarding both write endpoints
- Paginated list response with items + total + limit + offset
- Five passing TestClient tests
- README with run command, auth header instructions, and .env.example

Assignment 04 — Prompt Engineering Toolkit

Difficulty	Marks	Est. Hours	Required Stack
MEDIUM	10 marks	~2.5 hrs	Python · OpenAI (or equiv) API · python-dotenv
Pattern: GenAI Basics — LLM API calls, prompt principles, structured JSON output, token awareness			

Scenario

Before building agents, engineers need to feel how prompt wording changes model behaviour and how to get reliable, machine-parseable output. You are building a small toolkit that runs the same task three ways — naive, structured, and few-shot — over a set of inputs, and reports both the results and the token cost. This is the single most transferable skill into the MAS program.

What You Need to Build

A Python script, `prompt_toolkit.py`, that performs one concrete task — extracting structured fields from messy free-text bug reports — using three prompting strategies, and compares them. The API key is loaded from a `.env` file.

The task

Given a free-text bug report (e.g. “the app crashes when I hit save on the settings page on my iphone, super urgent, happens every time”), extract a structured record: {summary: str, component: str, severity: one of low/medium/high/critical, reproducible: bool}.

Three strategies (each its own function)

- **naive_extract(report: str) -> str:** A minimal one-line prompt. Returns whatever the model says (likely unstructured).
- **structured_extract(report: str) -> dict:** A prompt with a clear role, explicit field definitions, allowed severity values, and an instruction to return only JSON. Parses the JSON into a dict.
- **fewshot_extract(report: str) -> dict:** Builds on the structured prompt by adding 2–3 worked examples (input report → ideal JSON) before the real input. Parses to a dict.

Behaviours & constraints

- **Inputs:** Run all three strategies over the same set of at least 5 sample reports committed in a `reports.json` file.
- **JSON robustness:** `structured_extract` and `fewshot_extract` must handle the case where the model returns slightly malformed JSON — catch the parse error and retry once with a corrective instruction, or report the failure cleanly. Never crash.
- **Token reporting:** For each call, print the prompt and completion token counts from the API response usage field, and a running total.
- **Parameters:** Use `temperature=0` for the extraction calls and explain in the README why low temperature suits this task.

- **Comparison:** Produce a short comparison in the README: which strategy produced valid structured output most reliably across the 5 reports, and at what token cost.

Milestones

#	Phase	What You're Building	Time
M1	API Setup & Data	Load the key from .env, make a first successful call, and commit the reports.json with 5+ sample reports.	25 min
M2	Three Strategies	Implement the naive, structured, and few-shot extraction functions with their distinct prompts.	50 min
M3	Robust Parsing & Tokens	Add JSON-parse error handling with one corrective retry; print token usage and a running total.	35 min
M4	Comparison & Docs	Run all strategies over the 5 reports; write the README comparison of reliability vs token cost.	30 min

Marking Rubric (10 marks)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
1. Structure & Correctness	Three distinct strategy functions; key loaded from .env; calls succeed; JSON parsed into dicts	Strategies present but two prompts nearly identical, or key hardcoded (–1)	Calls fail; no working strategies; or only one strategy
2. Core Functionality (Prompting)	Structured and few-shot prompts use role, explicit fields, allowed values, and (few-shot) worked examples; produce valid structured output	Prompts work but lack explicit field/value constraints, or few-shot has no real examples	Prompts produce unstructured output; no real prompt engineering
3. Robustness (JSON + tokens)	Malformed JSON caught with a corrective retry; never crashes; token usage printed with running total	Handles JSON or tokens but not both; one crash path remains	No JSON error handling; crashes on bad output; no token reporting
4. End-to-End Run	All three strategies run over all 5 reports and produce the documented comparison	Runs over most inputs; comparison thin or missing one strategy	Crashes or only runs on one input
5. Documentation	README explains temperature choice +	Runs; README missing comparison	No README; no sample data

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
	strategy comparison (reliability vs token cost); reports.json committed	or temperature rationale	

Submission Checklist

- Three strategy functions with visibly distinct prompts
- API key loaded from .env (never committed); .env.example present
- JSON-parse error handling with one corrective retry
- Token usage printed per call with a running total
- reports.json with 5+ samples and a README comparison of reliability vs token cost

Assignment 05 — LLM Summariser & Classifier Service

Difficulty	Marks	Est. Hours	Required Stack
MEDIUM	10 marks	~3 hrs	Python · FastAPI · OpenAI (or equiv) · Pydantic · pytest
Pattern: GenAI + FastAPI — LLM-backed endpoints, structured output, auth, prompt design, tests			

Scenario

This capstone ties the three tracks together. A team wants an internal service that takes raw text and returns a clean summary and a category tag, behind a secured endpoint — the exact shape of an MAS agent tool. You are combining everything: a FastAPI service (Tracks 1–2) wrapping LLM calls with well-engineered, structured prompts (Track 3), with auth and tests.

What You Need to Build

A FastAPI application exposing two LLM-backed endpoints plus a health check. Requests require an API-key header (reuse the dependency pattern from Assignment 03). The model is called with structured prompts that return JSON, which is validated into Pydantic models before responding. The LLM key is loaded from the environment.

Endpoints

Method	Path	Auth	Description
GET	/health	None	Liveness check.
POST	/summarise	Required	Body: {text: str}. Returns {summary: str, word_count: int} — summary is 3 sentences max.
POST	/classify	Required	Body: {text: str}. Returns {category: one of bug/feature/question/feedback, confidence: float, rationale: str}.

Behaviours & constraints

- **Structured prompts:** Both endpoints instruct the model to return JSON only, with explicit fields and (for classify) the allowed category values. The response is parsed and validated into a Pydantic model before being returned.
- **LLM failure handling:** If the model returns invalid JSON, retry once with a corrective instruction; if it still fails, return HTTP 502 with a clear detail — do not return a 500 traceback.
- **Auth:** Both POST endpoints are guarded by an X-API-Key Depends dependency; missing/wrong key → 401.
- **Validation:** Empty text → 422 via Pydantic. classify must only ever return one of the four allowed categories — reject/repair anything else.
- **Secrets:** LLM key from environment; never committed; .env.example provided.

- **Tests:** pytest tests with TestClient: unauthed call → 401; empty text → 422; a mocked or live summarise returns the right shape. (Mocking the LLM call is encouraged so tests don't depend on the network.)

Sample requests to cover in README

- POST /summarise with a long paragraph → 200 with a ≤3-sentence summary and word_count
- POST /classify with a bug report → 200 with category 'bug' and a confidence
- POST /classify with no key → 401
- POST /summarise with empty text → 422

Milestones

#	Phase	What You're Building	Time
M1	Service & Models	FastAPI app, /health, Pydantic request/response models, and the env-loaded LLM client.	30 min
M2	LLM Endpoints	Implement /summarise and /classify with structured JSON prompts parsed into the response models.	50 min
M3	Auth & Failure Handling	Add the X-API-Key dependency to both POSTs; add JSON-retry and 502 mapping; cap categories to the allowed set.	40 min
M4	Tests & Docs	Write the TestClient tests (mock the LLM); README with run command, auth, and the four sample requests.	40 min

Marking Rubric (10 marks)

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
1. Structure & Correctness	FastAPI app; both LLM endpoints return Pydantic-validated structured output; key from environment; appears in /docs	Endpoints work but return raw model text instead of validated models, or key hardcoded (−1)	Not a working FastAPI app; LLM calls absent or broken
2. Core Functionality (Prompts + Auth)	Structured prompts yield valid JSON for both tasks; classify restricted to 4 categories; both POSTs auth-guarded via Depends	One endpoint solid, the other unreliable; or auth inline rather than a dependency	Prompts unstructured; no auth; classify returns arbitrary categories
3. Robustness (LLM + validation)	Invalid JSON → one corrective retry then 502; empty text → 422;	Some handled; one path returns a 500 or leaks a traceback	No failure handling; bad model output

Criterion	2 Marks — Full	1 Mark — Partial	0 Marks — Missing
	bad category repaired/rejected; no tracebacks		crashes the endpoint
4. End-to-End Run	All four README sample requests behave exactly as specified	3 of 4 behave correctly	Service crashes or core requests fail
5. Documentation & Tests	TestClient tests pass (LLM mocked); README with run + auth + 4 samples; .env.example committed	Runs; tests partial or README thin	No tests; no README; or key committed (red flag)

Submission Checklist

- FastAPI app with /health, /summarise, /classify
- Structured JSON prompts parsed into Pydantic response models
- X-API-Key Depends dependency on both POST endpoints
- JSON-retry then 502 on persistent LLM failure; classify capped to 4 categories
- TestClient tests (LLM mocked) + README with run, auth, and 4 sample requests + .env.example

Submission Guidelines

Artifact	Description
Source code	Python files. Single run command. No undocumented setup.
requirements.txt	Pinned versions including fastapi, uvicorn, pytest, and the LLM SDK where used.
README.md	Setup steps · run command · sample requests/outputs · any comparison the assignment asks for.
Sample data	.json fixtures (tasks.json, reports.json) committed — evaluators must not create data manually.
.env.example	Lists every required environment variable. The real .env is never committed.
Tests	pytest tests where the assignment requires them; all passing.

Code Quality Standards

- PEP-8 compliant; functions and variables type-hinted.
- Secrets loaded from the environment via .env — never committed.
- No raw tracebacks reach the user; errors map to clear messages or correct HTTP status codes.
- Pydantic models used for all FastAPI request and response bodies.

Grading Scale

Tier	Score	What It Looks Like
GOLD	45–50	All requirements met; clean structure; correct status codes; tests pass; thorough README.
SILVER	38–44	Required functionality correct; working end-to-end; good README; minor gaps.
BRONZE	30–37	Core functionality present and runs on the sample; basic README; some error paths or tests missing.
BELOW PASS	0–29	Required functionality absent or broken; service/script does not run end-to-end.

Pass criteria: 30/50 overall, with at least 12/30 across Assignments 01–03 and at least 8/20 across Assignments 04–05.

Engineers who pass this foundation course are ready to begin MAS-TRAINING-001, Module 1.